

Project Title
(Full-Text Search Engine on a Suffix-Array Index)

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Sai Krishna - 2520090156
Navaneeth – 2520090218
CHAITANYA - 2520090099

Team Number: 3
Section: 13

Under the guidance of

<Dyuti Banerjee>

<Professor ,CS/IT>



Hyderabad-500090, Telangana, India.

ABSTRACT

This project develops a searchable index for a collection of documents that supports efficient substring, prefix, and phrase queries with ranked results. The system uses an **$O(n \log^2 n)$ suffix array construction algorithm** to organize document suffixes and the **Kasai algorithm** to construct the Longest Common Prefix (LCP) array. LCP-accelerated binary search is implemented to reduce unnecessary character comparisons and improve query performance. The system retrieves matching document positions and ranks results based on their relevance and frequency. To evaluate the effectiveness of the proposed approach, its performance is compared with a traditional inverted index using metrics such as **query latency, index size, and search efficiency**. The comparison highlights the advantages and limitations of suffix-array-based indexing for full-text search. The project demonstrates that suffix arrays provide a compact and flexible structure for handling substring and phrase searches while maintaining efficient query processing.

Introduction

With the rapid growth of digital documents, efficient and flexible text searching has become essential for information retrieval systems. Traditional search systems commonly use **inverted indexes**, which map words to the documents in which they occur. They provide fast keyword and prefix-based searches, but substring and phrase queries can require additional processing, increasing query time and index complexity. Other approaches, such as naive string matching, scan large portions of the text for every query and become inefficient as the document collection grows.

To address these limitations, this project uses a **suffix array-based indexing approach**. Suffix arrays organize all suffixes of the document collection in sorted order, enabling efficient substring and prefix searching through binary search. The **Kasai algorithm** is used to construct the LCP array, while LCP-accelerated binary search reduces unnecessary character comparisons. The project also compares the proposed approach with an inverted index based on query latency, search performance, and index size.

Software and hardware requirements

Software Requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Programming Language:** Python 3.x / C++17
- **IDE/Editor:** Visual Studio Code, PyCharm, or any suitable code editor
- **Libraries/Tools:** Standard libraries for string processing, sorting, file handling, and data structures
- **Database/File Storage:** Text files or a suitable document storage system for the document collection
- **Testing Tools:** Command-line terminal and performance measurement tools
- **Version Control:** Git and GitHub (optional)

Hardware Requirements

- **Processor:** Intel Core i3 or equivalent and above
- **RAM:** Minimum 4 GB; 8 GB recommended
- **Storage:** Minimum 10 GB of free disk space
- **Display:** 1366 × 768 resolution or higher
- **Keyboard and Mouse:** Standard keyboard and pointing device
- **Internet:** Required for downloading software, libraries, and optional GitHub integration

These requirements are sufficient for implementing the suffix-array-based searchable index, testing substring, prefix, and phrase queries, and comparing its performance with an inverted index.

Signature of the Guide

Course Instructor